COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 21, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Mendoza, Allyson Mendoza, Nathaniel	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 **Algorithm Complexity**

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Merge Sort	*O (n log N)*	*O (n log N)*	Its best and worst case is both O(n log N) as merge sort works by dividing the array into smaller parts then merges them back. Therefore, this sorting algorithm continues and remains consistent regardless of their initial arrangement of values.
Shell Sort	*O (n log N)*	*O* (N^2)	This sorting algorithm uses a gap wherein this gap allows the values to be compared and swapped. For the best case, the values are already nearly sorted but in the worst case, the values are arranged in an unfavorable order. Therefore, it still depends on the gap.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Merge	Shell Sort
Best Case	***O (n log N)*** Input Size = 15 `Merge Sort Time: 21100 ns`	***O (n log N)*** Input Size = 15 `Shell Sort Time: 900 ns`

Worst Case	***O (n log N)***
	Input Size = 100 and 1000
	`Merge Sort Time: 32800 ns`
	`Merge Sort Time: 296200 ns`
		O (N^2)
		Input Size = 100 and 1000
		`Shell Sort Time: 9700 ns`
		`Shell Sort Time: 147100 ns`

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

EXPERIMENTAL
Header File

```cpp
// ILO A.1 (Shell Sort)
template <typename T>
void shellSort(T arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

```cpp
// ILO A.2 (Merge Sort)
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);
        merge(arr, left, middle, right);
    }
}
```

```cpp
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T *L = new T[n1];
    T *R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

Main Program

```cpp
#include <iostream>
#include <string>
#include <chrono>
#include <algorithm>
#include <functional>
#include "procedure.h"

const int SIZE = 100;

void displayArr(const int arr[], int size);

void timeDuration(const std::string& name,
                  const int original[],
                  int size,
                  std::function<void(int[])> sortFunc);

int main() {

    int arr[SIZE];

    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    for (int i = 0; i < SIZE; i++) {
        arr[i] = std::rand() % 1000 + 1;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(arr, SIZE);
    std::cout << "\n";

    // Performance comparison for Problem 2
    timeDuration("Shell Sort", arr, SIZE, [](int a[]) {
        shellSort(a, SIZE);
    });

    // Performance comparison for Problem 2 (Merge Sort)
    timeDuration("Merge Sort", arr, SIZE, [](int a[]) {
        mergeSort(a, 0, SIZE - 1);
    });

    return 0;
}

void displayArr(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

void timeDuration(const std::string& name,
                  const int original[],
                  int size,
                  std::function<void(int[])> sortFunc)
{
    std::vector<int> tempArr(original, original + size);

    auto start = std::chrono::high_resolution_clock::now();
    sortFunc(tempArr.data());
    auto stop = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(stop - start);

    std::cout << name << " Time: " << duration.count() << " ns\n";
    std::cout << "Sorted: ";
    displayArr(tempArr.data(), size);
    std::cout << "\n";
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE

```
START
    SET inputSizes = [1,000, 10,000, 100,000]
    FOR each N in inputSizes DO
        CREATE an array of size N
        FILL the array with values from 1 to N
        SET target = N + 1

        // Binary Search
        START timer
        SET low = 0
        SET high = N - 1
        WHILE low <= high DO
            SET mid = (low + high) / 2

            IF array[mid] == target THEN
                BREAK
            ELSE IF array[mid] < target THEN
                SET low = mid + 1
```

```
            ELSE
                SET high = mid - 1
            END IF
        END WHILE

        STOP timer
        RECORD Binary Search elapsed time

        // Linear Search
        START timer

        FOR i = 0 TO N - 1 DO
            IF array[i] == target THEN
                BREAK
            END IF
        END FOR

        STOP timer
        RECORD Linear Search elapsed time

        DISPLAY input size N
        DISPLAY Binary Search elapsed time
        DISPLAY Linear Search elapsed time
    END FOR
END
```

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Binary Search Index Result: 98 Binary Search Execution Time: 600 nanoseconds	Linear Search Index Result: 98 Linear Search Execution Time: 900 nanoseconds
10,000	Binary Search Index Result: 9998 Binary Search Execution Time: 500 nanoseconds	Linear Search Index Result: 9998 Linear Search Execution Time: 49100 nanoseconds
100,000	Binary Search Index Result: 99998 Binary Search Execution Time: 800 nanoseconds	Linear Search Index Result: 99998 Linear Search Execution Time: 503500 nanoseconds

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>
#include <functional>

int linearSearch(const std::vector<int>& arr, int target);
int binarySearch(const std::vector<int>& arr, int target);
void displayArr(const std::vector<int>& arr, int size);
void timeSearch(const std::string& name, const std::vector<int>& arr, int target, std::function<int(const std::vector<int>&, int)> searchFunc) ;

int main() {
    const int SIZE = 1000;
    std::vector<int> data(SIZE);
    std::iota(data.begin(), data.end(), 1);
    int target = 99;

    std::cout << "Target: " << target << " in " << SIZE << " elements.\n\n";

    // Benchmark using your functions
    timeSearch("Linear Search", data, target, linearSearch);
    timeSearch("Binary Search", data, target, binarySearch);

    return 0;
}

int linearSearch(const std::vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); ++i) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

```cpp
int binarySearch(const std::vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void displayArr(const std::vector<int>& arr, int size) {
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

void timeSearch(const std::string& name,
                const std::vector<int>& arr,
                int target,
                std::function<int(const std::vector<int>&, int)> searchFunc)
{
    auto start = std::chrono::high_resolution_clock::now();
    int result = searchFunc(arr, target);
    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(
        std::chrono::high_resolution_clock::now() - start).count();

    std::cout << name << " -> Index: " << result << " | Time: " << duration << " ns\n";
}
```

Analysis
Based on the given results, it shows that the execution time of Binary Search is much faster than the execution time of Linear Search. For an input size of 1,000, the Binary Search takes only 600 nanoseconds while Linear Search takes 900 nanoseconds. So, as you observed, the more you increase the input size the Binary Search only takes a few seconds as it is divided by half and another half. But, Linear Search takes a lot of time as it searches one-by-one. Therefore, Binary Search is more efficient for searching large and sorted data while, LinearSearch is simpler to use when the data is small and unsorted.

Mendoza, Allyson (CONCLUSION)
In conclusion, this hands-on activity, I learned that analyzing algorithm complexity is important in determining the efficiency of different algorithms. I saw that Merge Sort is much more consistent rather than Shell Sort. I also observed that Binary Search is much faster and efficient to use when it comes to large data rather than Linear Search as its execution time takes a lot of time. Therefore, understanding the best and worst case helps us to choose the most suitable algorithm depending on its input size.
Mendoza, Nathaniel (CONCLUSION)
After performing today's laboratory activity, I was able to recall the use and importance of chrono time library for this hands-on activity specially when I need to compare and contrast the amount of time needed from specific sorting algorithms like the one we used for our procedure which is the Shell and Merge Sort which proves that Shell is faster than Merge sort as it has faster sorting technique than Merge. Additionally, the time taken from both the best and worst case of the said algorithms shows that Merge raises its time complexity exponentially or basically O(n log n) which is higher than Shell Sort thats only $O(N^2)$.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.